TÀI LIỆU KỸ THUẬT CHUYÊN SÂU

Dự án: E-Nanny – Smart Baby Monitor
Kiến trúc: STM32F401CCU6 (MCU chính) + ESP32 (IoT Gateway)
Phiên bản: 1.0 — Biên soạn cho người mới bắt đầu
Phạm vi: Phân tích code, thiết kế module, giao thức truyền thông và quy trình triển khai

MỤC LỤC

1. Chương 1 — Giới thiệu hệ thống: Sơ đồ khối và nguyên lý hoạt động
2. Chương 2 — Cấu hình phần cứng: Pinout, Timer và tài nguyên UART
3. Chương 3 — Phân tích Driver chi tiết (DHT22 & DFPlayer)
4. Chương 4 — Tư duy lập trình hệ thống: State Machine và Non-blocking
5. Chương 5 — Hướng dẫn cài đặt, chạy thực tế và debug

CHƯƠNG 1 — GIỚI THIỆU HỆ THỐNG

1.1. Tổng quan dự án

E-Nanny (Smart Baby Monitor) là một hệ thống giám sát và chăm sóc trẻ sơ sinh từ xa. Thiết bị thực hiện bốn nhóm chức năng chính: đo **nhiệt độ và độ ẩm** môi trường, **rung nôi** tự động với biên độ an toàn, **phát nhạc ru** theo yêu cầu, và **truyền dữ liệu** lên ứng dụng di động thông qua kết nối WiFi/MQTT do ESP32 đảm nhiệm.

Kiến trúc phần cứng chia trách nhiệm rất rõ ràng: **STM32F401CCU6** chịu trách nhiệm về phần thời gian thực (real-time) — điều khiển servo rung nôi, đọc cảm biến, giao tiếp với module MP3 — trong khi **ESP32** đảm nhiệm kết nối Internet và giao diện với người dùng. Hai vi điều khiển trao đổi dữ liệu qua một đường UART duy nhất sử dụng khung truyền tự thiết kế.

1.2. Sơ đồ khối hệ thống

```
                +----------------------+
                |   Smartphone App     |
                |   (MQTT Client)      |
                +----------+-----------+
                           |
                           | WiFi / MQTT
                           |
                +----------v-----------+
                |       ESP32          |
                |  (Gateway + WiFi)    |
                +----------+-----------+
                           |
                           | UART1 @ 115200 bps
                           | Frame: AA 55 LEN CMD DATA CRC
                           |
        +----------------------v------------------------+
        |               STM32F401CCU6                  |
        |                                              |
        |   +------------+   +-----------------+       |
        |   |  App Layer |   |  State Machine  |       |
        |   | (Logic nghiep vu) |  RX UART      |       |
        |   +------+-----+   +--------+--------+       |
        |          |                  |                |
        |          v                  v                |
        |   +-------------+   +---------------+        |
        |   | DHT22 Driver|   | DFPlayer Drv. |        |
        |   | (1-Wire)    |   | (UART2)       |        |
        |   +------+------+   +-------+-------+        |
        |          |                  |                |
        |   +------v----+     +-------v-----+          |
        |   | TIM2 PWM  |     |  UART2 TX   |          |
        |   | (Servo)   |     |             |          |
        |   +-----+-----+     +-------------+          |
        +---------|-------------------------+---------+
                  |                         |
           +------v------+           +-------v--------+
           | Servo SG90  |           |  DFPlayer Mini |
           | (Rung noi)  |           |  (MP3 + Loa)   |
           +-------------+           +----------------+

           +-------------+
           |  DHT22      |---- 1-Wire ----> STM32 PA4
           | (T & H)     |
           +-------------+
```

1.3. Nguyên lý hoạt động

Hệ thống hoạt động theo mô hình **super-loop non-blocking** — vòng lặp chính của STM32 liên tục quét ba nhóm công việc, nhưng mỗi nhóm đều tự quản lý thời gian của mình để không bao giờ chặn các nhóm còn lại. Cụ thể:

Nhóm 1 — Đọc cảm biến môi trường. Cứ mỗi 2 giây, STM32 kích hoạt driver DHT22 để lấy một mẫu nhiệt độ và độ ẩm. Sau khi đọc thành công, dữ liệu được đóng gói vào frame có CRC và gửi qua UART1 sang ESP32. ESP32 tiếp tục đẩy lên cloud thông qua MQTT.

Nhóm 2 — Nhận lệnh điều khiển từ ESP32. Khi người dùng bấm nút trên ứng dụng (ví dụ "Rung nôi", "Phát nhạc ru số 3"), ESP32 đóng gói thành frame và gửi xuống STM32. STM32 nhận từng byte bằng ngắt UART, ghép frame qua máy trạng thái, kiểm tra CRC, rồi thực thi lệnh ở vòng lặp chính.

Nhóm 3 — Điều khiển thiết bị ngoại vi. Khi có lệnh rung nôi, cờ `is_rocking` được bật và hàm `Servo_Rocking_Task()` sẽ tự động quay qua lại giữa hai góc 60° và 120° với bước 1° mỗi 20ms — không cần bất kỳ delay nào. Khi có lệnh phát nhạc, driver DFPlayer sẽ gửi lệnh điều khiển qua UART2 tới module MP3.

> **Điểm then chốt của thiết kế:** Ba nhóm trên không phải chạy tuần tự (xong cái này mới tới cái kia). Nhờ cơ chế `HAL_GetTick()` kết hợp với ngắt UART, cả ba luôn trong trạng thái "hoạt động đồng thời" ở góc nhìn của người dùng.

CHƯƠNG 2 — CẤU HÌNH PHẦN CỨNG

2.1. Bảng chân (Pinout) của STM32F401CCU6

Toàn bộ pinout của dự án được tổng hợp trong bảng dưới đây. Đây là phần **bắt buộc phải tuân thủ** khi đấu nối phần cứng để không phải sửa lại firmware.

Chân STM32	Chức năng	Chế độ (Mode)	Kết nối tới	Ghi chú
PA4	DHT22 Data	GPIO Output Push-Pull / Input (chuyển runtime)	Chân DATA của DHT22	Cần điện trở kéo lên 4.7kΩ lên VCC

Chân STM32	Chức năng	Chế độ (Mode)	Kết nối tới	Ghi chú
PA0	TIM2_CH1 (PWM)	Alternate Function PP	Chân tín hiệu Servo SG90	PWM 50Hz, độ rộng xung 0.5ms–2.5ms
PA9	USART1_TX	Alternate Function	RX của ESP32	115200-8-N-1
PA10	USART1_RX	Alternate Function (có Pull-up)	TX của ESP32	Bật NVIC interrupt
PA2	USART2_TX	Alternate Function	RX của DFPlayer Mini	9600-8-N-1
PA3	USART2_RX	Alternate Function	TX của DFPlayer Mini	Dùng để đọc phản hồi (tuỳ chọn)
PA13 / PA14	SWDIO / SWCLK	Serial Wire Debug	ST-Link hoặc J-Link	Không được dùng cho mục đích khác
3V3 / GND	Nguồn	—	Toàn hệ thống	DHT22 và Servo dùng 5V (qua module nguồn riêng)

2.2. Bảng tài nguyên Timer

Hai Timer được huy động cho hai mục đích hoàn toàn khác nhau. Đây là lý do phải chọn Timer có nguồn clock phù hợp trên từng bus APB.

Timer	Bus	Prescaler	Period (ARR)	Mode	Dùng cho	Giải thích tính toán
TIM2	APB1 (48 MHz sau ×2)	47	19999	PWM Generation CH1	Servo SG90 (rung nôi)	48MHz / (47+1) = 1MHz tick. Period 20000 → chu kỳ 20ms = 50Hz (chuẩn servo)

Timer	Bus	Prescaler	Period (ARR)	Mode	Dùng cho	Giải thích tính toán
TIM11	APB2 (84 MHz)	83	65535	Base Counter	Tạo delay chính xác theo µs cho DHT22	84MHz / (83+1) = 1MHz tick → 1 tick = 1µs

> **Nguyên tắc chọn Prescaler:** Công thức chuẩn là `Prescaler = (ClockBus / TickMongMuon) - 1`. Với TIM11 ta muốn tick 1µs trên bus 84MHz nên Prescaler = 84 - 1 = 83. Với TIM2 ta muốn tick 1µs trên bus 48MHz hiệu dụng nên Prescaler = 48 - 1 = 47.

2.3. Bảng tài nguyên UART

Ngoại vi	Baudrate	Word Length	Parity	Stop Bit	Chế độ	Mục đích
USART1	115200	8 bit	None	1	Interrupt RX, Polling TX	Giao tiếp với ESP32
USART2	9600	8 bit	None	1	Polling TX	Gửi lệnh cho DFPlayer Mini

2.4. Cấu trúc thư mục source code

Dự án được tổ chức theo kiến trúc phân tầng rõ ràng. Đây là cấu trúc **bắt buộc** để đảm bảo tính module hoá và tái sử dụng.

```
STM32_SmartBabyMonitor/
+-- Core/
|   +-- Inc/                    (Header he thong - CubeMX tu sinh)
|   |   +-- main.h
|   |   +-- stm32f4xx_hal_conf.h
|   |
|   +-- Src/                    (Source he thong - CubeMX tu sinh)
|   |   +-- main.c              (Chi goi Init va Loop cua App)
|   |   +-- system_stm32f4xx.c
|   |
|   +-- App/                    (Tu viet - tang ung dung)
|       +-- ChuongTrinhChinh.c  (Dieu phoi task, state machine UART)
```

```
|       +-- ChuongTrinhChinh.h
|       +-- Libraries/
|           +-- DHT22/
|           |   +-- DHT22.c      (Driver 1-Wire cho cam bien)
|           |   +-- DHT22.h
|           +-- DFPlayer/
|               +-- DFPlayer.c   (Driver giao thuc 10 byte)
|               +-- DFPlayer.h
|
+-- Drivers/                     (HAL + CMSIS cua ST)
```

CHƯƠNG 3 — PHÂN TÍCH DRIVER CHI TIẾT

3.1. Driver DHT22 — Phân tích từng hàm

3.1.1. Tại sao phải đổi chiều chân GPIO (Input ⇄ Output)?

DHT22 sử dụng giao thức **1-Wire tự chế** — chỉ dùng duy nhất một sợi dây tín hiệu cho cả việc MCU gửi lệnh "đánh thức" và việc DHT22 trả dữ liệu. Do đó, tại mỗi thời điểm, chỉ một bên được phép *lái* (drive) đường tín hiệu. Nếu cả hai cùng lái và một bên đẩy HIGH trong khi bên kia kéo LOW, sẽ xảy ra chập ngắn mạch cục bộ gây cháy GPIO.

Giải pháp: MCU phải chủ động chuyển chân PA4 sang chế độ **Output Push-Pull** khi cần gửi tín hiệu START (kéo LOW 1.2ms rồi HIGH 20µs), sau đó chuyển về chế độ **Input** (kèm pull-up ngoài) để "nghe" DHT22 trả lời.

```
// DHT22.c - Ham phu tro chuyen che do chan GPIO

static void Set_Pin_Output(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;   // Push-Pull de lai duoc ca HIGH va L
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
}

static void Set_Pin_Input(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin) {
```

```
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;       // Tha chan de chi "nghe"
    GPIO_InitStruct.Pull = GPIO_NOPULL;           // Dien tro keo len o ben ngoai (4.7k
    HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
}
```

3.1.2. Tại sao phải dùng Timer (TIM11) thay vì HAL_Delay()?

Hàm	Độ phân giải	Vấn đề
`HAL_Delay(ms)`	1 mili giây	DHT22 yêu cầu chính xác 40µs, 80µs — sai số quá lớn
Vòng `for` trống	Không xác định	Trình biên dịch tối ưu có thể xoá vòng lặp
TIM11 counter	1 µs	Chính xác, không phụ thuộc optimization level

```
// DHT22.c - Ham delay chinh xac toi microsecond
extern TIM_HandleTypeDef htim11;

static void delay_us(uint16_t us) {
    __HAL_TIM_SET_COUNTER(&htim11, 0);              // Reset bo dem ve 0
    while (__HAL_TIM_GET_COUNTER(&htim11) < us);    // Doi den khi dem du us tick
}
```

> **Lưu ý bắt buộc:** Trong `ChuongTrinh_Init()`, phải gọi `HAL_TIM_Base_Start(&htim11);` để kích hoạt bộ đếm trước khi bất kỳ hàm nào trong driver DHT22 được dùng. Bỏ qua bước này là nguyên nhân phổ biến nhất khiến driver "tưởng là đúng mà luôn đọc lỗi".

3.1.3. Giao thức 1-Wire của DHT22

```
MCU keo LOW 1.2ms    MCU keo HIGH 20us    MCU chuyen sang INPUT
    +--------+           +-----+                  +----->
____|        |___________|     |__________________|
    |                          |
    v                          v
  Start                      Release
                                        DHT22 tra loi:
                                        +-- 80us LOW (ACK) --+-- 80us HIGH (ACK) --+
                                        |                    |                       |
```

```
                                        +----+----+----+----+----+----+---- ...
                                        bit1 bit2 bit3 ...                 40 bit
```

Phân biệt bit 0 và bit 1: Mỗi bit bắt đầu bằng một xung LOW 50µs. Sau đó:

- Nếu xung HIGH kéo dài ~26µs → đó là bit **0** (xung ngắn)
- Nếu xung HIGH kéo dài ~70µs → đó là bit **1** (xung dài)

Thủ thuật đọc: sau cạnh lên, đợi đúng **40µs** rồi đọc mức logic. Nếu đã LOW thì xung HIGH ngắn hơn 40µs → bit 0. Nếu vẫn HIGH thì xung HIGH dài hơn 40µs → bit 1.

3.1.4. Hàm DHT22_Start() — Giải phẫu từng dòng

```
static uint8_t DHT22_Start(void) {
    uint8_t response = 0;
    uint8_t timeout = 0;

    // Buoc 1: MCU lai chan, keo LOW 1.2ms de danh thuc DHT22
    //   Chuan chinh thuc: >= 1ms. Ta chon 1.2ms de co bien an toan.
    Set_Pin_Output(DHT_PORT, DHT_PIN);
    HAL_GPIO_WritePin(DHT_PORT, DHT_PIN, GPIO_PIN_RESET);
    delay_us(1200);

    // Buoc 2: MCU keo HIGH 20us de ra hieu "da xong start"
    HAL_GPIO_WritePin(DHT_PORT, DHT_PIN, GPIO_PIN_SET);
    delay_us(20);

    // Buoc 3: Chuyen chan sang INPUT - tha quyen dieu khien cho DHT22
    //   Tu day MCU chi doc, khong duoc ghi nua
    Set_Pin_Input(DHT_PORT, DHT_PIN);
    delay_us(40);

    // Buoc 4: Kiem tra xung ACK cua DHT22
    //   DHT22 phai keo LOW 80us, roi HIGH 80us
    if (!(HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {        // Dang LOW
        delay_us(80);
        if ((HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {      // Da chuyen HIGH
            response = 1;                                 // ACK hop le
        }
    }

    // Cho het xung HIGH cuoi cua ACK (truoc khi bit du lieu dau tien toi)
    //   Co timeout de khong treo may neu cam bien hong
    while ((HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {
```

```
        delay_us(1);
        if (++timeout > 100) return 0;
    }
    return response;
}
```

Điểm đáng chú ý: Vòng `while` cuối cùng có biến `timeout`. Đây là *kỹ thuật sống còn* trong lập trình embedded — không bao giờ được có vòng lặp chờ tín hiệu mà không có điều kiện thoát. Nếu DHT22 bị hỏng phần cứng, chân luôn ở HIGH thì vòng while sẽ chạy mãi và treo hệ thống.

3.1.5. Hàm DHT22_Read_Byte()

```
static uint8_t DHT22_Read_Byte(void) {
    uint8_t i = 0;
    uint16_t timeout;

    // Doc 8 bit theo thu tu MSB first
    for (uint8_t j = 0; j < 8; j++) {

        // 1. Doi het phan LOW 50us dau moi bit
        timeout = 0;
        while (!(HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {
            delay_us(1);
            if (++timeout > 100) return 0;    // Timeout chong treo
        }

        // 2. Dang trong pha HIGH, doi dung 40us roi doc muc logic
        delay_us(40);

        if (!(HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {
            // Tai moc 40us ma da LOW --> xung HIGH ngan --> bit 0
            i &= ~(1 << (7 - j));
        } else {
            // Tai moc 40us van HIGH --> xung HIGH dai --> bit 1
            i |= (1 << (7 - j));

            // Doi phan con lai cua xung HIGH (70-40 = 30us nua)
            timeout = 0;
            while ((HAL_GPIO_ReadPin(DHT_PORT, DHT_PIN))) {
                delay_us(1);
                if (++timeout > 100) return 0;
            }
        }
```

```
    }
    return i;
}
```

3.1.6. Hàm public DHT22_Read() và kiểm tra Checksum

DHT22 gửi tổng cộng **40 bit = 5 byte** có cấu trúc:

Byte	Ý nghĩa
1	Humidity HIGH (8 bit cao của độ ẩm)
2	Humidity LOW (8 bit thấp của độ ẩm)
3	Temperature HIGH (8 bit cao của nhiệt độ)
4	Temperature LOW (8 bit thấp của nhiệt độ)
5	**Checksum** = (byte1 + byte2 + byte3 + byte4) & 0xFF

```
uint8_t DHT22_Read(volatile DHT22_Data_t *DHT_Data) {
    uint8_t Rh_byte1, Rh_byte2, Temp_byte1, Temp_byte2;
    uint16_t checkSum = 0;

    if (DHT22_Start() == 1) {
        Rh_byte1   = DHT22_Read_Byte();
        Rh_byte2   = DHT22_Read_Byte();
        Temp_byte1 = DHT22_Read_Byte();
        Temp_byte2 = DHT22_Read_Byte();
        checkSum   = DHT22_Read_Byte();
    }

    // Kiem tra tong bit - chong nhieu duong truyen
    if (checkSum == ((Rh_byte1 + Rh_byte2 + Temp_byte1 + Temp_byte2) & 0xFF)) {
        uint16_t raw_temp  = (Temp_byte1 << 8) | Temp_byte2;
        uint16_t raw_humid = (Rh_byte1   << 8) | Rh_byte2;

        // DHT22 dong goi nhiet va am nhan 10 --> chia lai 10
        DHT_Data->Temperature = (float)raw_temp  / 10.0f;
        DHT_Data->Humidity    = (float)raw_humid / 10.0f;
        return 1;       // Doc thanh cong
    }
```

```
    return 0;           // Loi checksum --> bo qua mau nay
}
```

> **Triết lý "fail safe":** Khi checksum sai, hàm không trả giá trị đoán mò mà trả về 0 (lỗi). App layer sẽ tự quyết định có dùng lại mẫu cũ hay không. Đây là nguyên tắc "*trả đúng hoặc không trả gì cả*" trong lập trình embedded.

3.2. Giao thức UART tự thiết kế (STM32 ⇄ ESP32)

3.2.1. Tại sao không gửi text thô?

Gửi chuỗi dạng `"T=28.5,H=65\n"` là cách đơn giản nhất, nhưng trong sản phẩm thật sự sẽ gây ra nhiều rắc rối: nếu mất một byte giữa đường (nhiễu điện từ, đứt tạm thời dây kết nối) thì sẽ không biết cách khôi phục; không phát hiện được dữ liệu bị bóp méo; và rất khó mở rộng khi muốn thêm lệnh mới.

Giải pháp chuyên nghiệp: Thiết kế một **khung truyền nhị phân (binary frame)** có header cố định, độ dài tường minh và mã kiểm tra lỗi.

3.2.2. Cấu trúc Frame

```
+-------+-------+--------+-------+-----------------+-------+
| 0xAA  | 0x55  | LENGTH |  CMD  |  DATA (N byte)  |  CRC  |
+-------+-------+--------+-------+-----------------+-------+
   H1      H2              Lenh      Noi dung        XOR
```

Trường	Kích thước	Mục đích
Header 1 (0xAA)	1 byte	Đánh dấu bắt đầu frame
Header 2 (0x55)	1 byte	Xác nhận kép, chống nhiễu ngẫu nhiên
Length	1 byte	Số byte của (CMD + DATA)
CMD	1 byte	Mã lệnh điều khiển
Data	N byte	Tham số của lệnh
CRC	1 byte	XOR của toàn bộ (Length + CMD + Data)

3.2.3. Bảng lệnh giao tiếp (Protocol Table)

CMD	Hướng	Ý nghĩa	Cấu trúc Data
`0x01`	ESP32 → STM32	Phát / Dừng nhạc ru	1 byte: 0 = Stop, 1..N = Track number
`0x02`	ESP32 → STM32	Quay servo đến góc cố định	1 byte: 0–180 (độ)
`0x03`	2 chiều	Gửi / Yêu cầu nhiệt độ-độ ẩm	2 byte: Temp, Humidity
`0x04`	ESP32 → STM32	Bật / Tắt rung nôi	1 byte: 0 = Tắt, 1 = Bật

3.2.4. Ví dụ frame thực tế

Yêu cầu ESP32 bảo STM32 phát nhạc track số 5:

```
AA 55 02 01 05 06
|  |  |  |  |  +-- CRC = 0x02 XOR 0x01 XOR 0x05 = 0x06
|  |  |  |  +----- Data = 0x05 (track 5)
|  |  |  +-------- CMD  = 0x01 (play)
|  |  +----------- LEN  = 0x02 (CMD + Data)
|  +-------------- H2   = 0x55
+---------------- H1   = 0xAA
```

CHƯƠNG 4 — TƯ DUY LẬP TRÌNH HỆ THỐNG

4.1. Máy trạng thái (State Machine) trong ngắt UART

4.1.1. Vấn đề với HAL_UART_Receive() blocking

Hàm `HAL_UART_Receive(&huart1, buf, 10, 1000)` sẽ chặn CPU tối đa 1 giây để chờ nhận đủ 10 byte. Trong suốt 1 giây đó, servo không quay được, DHT22 không đọc được, hệ thống "đơ" hoàn toàn. Đây là **anti-pattern tuyệt đối không được dùng** trong firmware sản phẩm.

4.1.2. Giải pháp: Nhận từng byte qua ngắt + State Machine

Thay vì chờ cả frame, ta chỉ đăng ký ngắt để nhận đúng **1 byte**. Mỗi khi byte đó đến, ISR sẽ chạy rất nhanh (vài µs), ghép byte đó vào buffer dựa trên trạng thái hiện tại của máy trạng thái, rồi ngay lập tức đăng ký ngắt cho byte kế tiếp. CPU gần như luôn rảnh để làm việc khác.

4.1.3. Sơ đồ máy trạng thái

```
           0xAA            0x55           LEN byte      Payload + CRC
[WAIT_H1] -----> [WAIT_H2] -----> [WAIT_LEN] -----> [WAIT_PAYLOAD] ---+
     ^                                                                |
     |       (byte sai o bat ky trang thai nao hoac CRC khong khop)   |
     +----------------------------------------------------------------+
                           (tra ve trang thai chinh)
```

Bốn trạng thái của máy có vai trò như một "người soát vé" nhiều lớp: nếu byte đầu không phải `0xAA`, không chuyển trạng thái; nếu byte thứ hai không phải `0x55`, reset về đầu; nếu Length quá lớn, reset về đầu (chống tràn buffer); cuối cùng nếu CRC không khớp, cũng reset. Bất kỳ sai lệch nào cũng đưa máy về đúng trạng thái chờ ban đầu, đảm bảo hệ thống *tự đồng bộ lại* khi có nhiễu.

4.1.4. Triển khai state machine

```
// ChuongTrinhChinh.c - Bien global can volatile vi duoc dung trong ngat
volatile uint8_t  rx_byte;                  // Bo dem 1 byte moi lan ngat
volatile uint8_t  rx_buffer[16];            // Bo dem chua frame ghep
volatile uint8_t  rx_index = 0;
volatile uint8_t  frame_len = 0;
volatile uint8_t  frame_ready = 0;          // Co bao frame da xong

typedef enum {
    WAIT_HEADER_1,
    WAIT_HEADER_2,
    WAIT_LENGTH,
    WAIT_PAYLOAD_AND_CRC
} RX_STATE;

volatile RX_STATE rx_state = WAIT_HEADER_1;

// Callback duoc HAL tu dong goi khi nhan du 1 byte
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
```

```
        switch (rx_state) {
            case WAIT_HEADER_1:
                if (rx_byte == 0xAA) rx_state = WAIT_HEADER_2;
                break;

            case WAIT_HEADER_2:
                if (rx_byte == 0x55) rx_state = WAIT_LENGTH;
                else                 rx_state = WAIT_HEADER_1;  // Reset
                break;

            case WAIT_LENGTH:
                frame_len = rx_byte;
                rx_index = 0;
                // Kiem tra LEN hop le de tranh tran buffer
                if (frame_len > 0 && frame_len < 12)
                    rx_state = WAIT_PAYLOAD_AND_CRC;
                else
                    rx_state = WAIT_HEADER_1;
                break;

            case WAIT_PAYLOAD_AND_CRC:
                rx_buffer[rx_index++] = rx_byte;
                if (rx_index == (frame_len + 1)) {  // Du payload + 1 byte CRC
                    // Tinh CRC XOR de so sanh
                    uint8_t calc_crc = frame_len;
                    for (int i = 0; i < frame_len; i++)
                        calc_crc ^= rx_buffer[i];

                    if (calc_crc == rx_buffer[frame_len])
                        frame_ready = 1;         // Bao main loop xu ly
                    rx_state = WAIT_HEADER_1;
                }
                break;
        }

        // QUAN TRONG: Re-arm ngat cho byte tiep theo
        HAL_UART_Receive_IT(&huart1, (uint8_t*)&rx_byte, 1);
    }
}
```

4.1.5. Xử lý lệnh ở vòng lặp chính (không nằm trong ISR)

```c
void Process_Command(void) {
    uint8_t command = rx_buffer[0];
    uint8_t data    = rx_buffer[1];

    switch (command) {
        case 0x01:  // Phat nhac
            if (data > 0) DFPlayer_PlayTrack(data);
            else          DFPlayer_Stop();
            break;

        case 0x02:  // Servo quay goc co dinh
            if (data <= 180) {
                is_rocking = 0;
                Set_Servo_Angle(data);
            }
            break;

        case 0x04:  // Bat/Tat rung noi
            is_rocking = (data == 1) ? 1 : 0;
            if (!is_rocking) Set_Servo_Angle(90);
            break;
    }
}
```

Checklist an toàn khi dùng ngắt UART:

- Buffer luôn có kiểm tra giới hạn (`frame_len < 12`) để chống overflow
- Có cơ chế reset về `WAIT_HEADER_1` khi gặp byte sai
- Luôn re-arm `HAL_UART_Receive_IT()` ở cuối callback
- Mọi biến dùng chung giữa ISR và `main()` đều phải khai báo `volatile`
- Tuyệt đối không gọi `HAL_Delay` , `printf` , logic dài trong callback

4.2. Đa nhiệm không cần RTOS — Non-blocking Super Loop

4.2.1. Vấn đề của HAL_Delay()

Giả sử ta muốn chạy đồng thời ba tác vụ: đọc DHT22 mỗi 2 giây, rung nôi mỗi 20ms, phản hồi UART ngay lập tức. Nếu viết kiểu blocking:

```
// ANTI-PATTERN: Khong bao gio lam nhu the nay
while (1) {
    DHT22_Read(&data);
    HAL_Delay(2000);       // TREO 2 giay - khong lam gi khac duoc
    Servo_Rotate();
    HAL_Delay(20);
}
```

Trong 2 giây chờ đó, servo đứng im, UART không phản hồi. Đây là sai thiết kế *cơ bản*.

4.2.2. Mẫu thiết kế Cooperative Scheduling

Hàm `HAL_GetTick()` trả về số mili giây từ lúc khởi động (chạy bằng SysTick nền, không chặn CPU). Ta biến mỗi tác vụ thành một hàm tự kiểm tra "đã đến giờ mình chạy chưa?":

```
void Task_A(void) {
    static uint32_t last_time = 0;
    if (HAL_GetTick() - last_time >= INTERVAL_MS) {
        last_time = HAL_GetTick();
        // ... cong viec cua Task A ...
    }
}
```

Biến `static` giữ giá trị qua nhiều lần gọi. Không có `HAL_Delay`. CPU trả quyền ngay nếu chưa đến hạn. Gọi liên tiếp `Task_A()`, `Task_B()`, `Task_C()` trong `while(1)` → cả ba chạy song song ở góc nhìn của người dùng.

4.2.3. Case study: Servo_Rocking_Task()

Yêu cầu: Servo quay qua lại giữa 60° và 120° (biên độ an toàn, không lật nôi), mỗi bước 1° cách nhau 20ms cho chuyển động mượt như ru thật, có thể bật/tắt từ ESP32 bất cứ lúc nào.

```
// ChuongTrinhChinh.c - Bien trang thai
uint8_t  is_rocking = 0;        // 1 = dang rung, 0 = dung
int16_t  current_angle = 90;    // Goc hien tai (bat dau o giua)
int8_t   rock_dir = 1;          // Huong quay: +1 = tang, -1 = giam
uint32_t last_rock_time = 0;

void Servo_Rocking_Task(void) {
    // Khong rung --> thoat ngay, khong ton CPU
    if (!is_rocking) return;
```

```
    // Kiem tra da du 20ms tu lan dich chuyen truoc chua
    if (HAL_GetTick() - last_rock_time >= 20) {
        last_rock_time = HAL_GetTick();

        current_angle += rock_dir;       // Tang / giam 1 do

        // Chan gioi han an toan
        const int16_t SAFE_MAX = 120;
        const int16_t SAFE_MIN = 60;

        if (current_angle >= SAFE_MAX) {
            current_angle = SAFE_MAX;
            rock_dir = -1;               // Dao chieu
        } else if (current_angle <= SAFE_MIN) {
            current_angle = SAFE_MIN;
            rock_dir = 1;
        }

        Set_Servo_Angle(current_angle);
    }
}
```

4.2.4. So sánh hai cách tiếp cận

Tiêu chí	Blocking (HAL_Delay)	Non-blocking (HAL_GetTick)
Thời gian phản hồi lệnh mới	Có thể > 1 giây	< 1 ms
CPU khi rảnh	100% (bận chờ delay)	Gần 0%
Thêm task mới	Phức tạp, phải tính lại timing	Chỉ cần thêm hàm `Task_X()`
Debug từng bước	Khó (blocking)	Dễ (bước qua được)

4.2.5. Mẫu tổng hợp — Super Loop hoàn chỉnh

```
void ChuongTrinh_Loop(void) {
    static uint32_t last_read_time = 0;

    // Task 1: Doc DHT22 moi 2 giay
    if (HAL_GetTick() - last_read_time >= 2000) {
        last_read_time = HAL_GetTick();
```

```
        if (DHT22_Read(&Nanny_EnvData) == 1) {
            Send_DHT22_To_ESP32(Nanny_EnvData.Temperature,
                                Nanny_EnvData.Humidity);
        }
    }

    // Task 2: Xu ly lenh UART khi co frame moi
    if (frame_ready) {
        Process_Command();
        frame_ready = 0;
    }

    // Task 3: Rung noi (tu quan ly thoi gian ben trong)
    Servo_Rocking_Task();
}
```

> **Khi nào cần lên FreeRTOS?** Super-loop non-blocking đủ cho các hệ thống dưới 5 task với timing không quá ngặt. Khi cần ưu tiên task, timing chính xác ở mức vi giây, hoặc có task blocking dài như TCP/IP, SD card — lúc đó hãy cân nhắc FreeRTOS. Với E-Nanny ở quy mô hiện tại, super-loop là đủ và *đơn giản hơn rất nhiều*.

CHƯƠNG 5 — HƯỚNG DẪN CÀI ĐẶT & DEBUG

5.1. Yêu cầu phần mềm

Công cụ	Phiên bản tối thiểu	Mục đích
STM32CubeIDE	1.13.0	IDE chính, có tích hợp CubeMX + GCC + OpenOCD
STM32CubeMX	6.8.0 (đã bundle trong IDE)	Cấu hình ngoại vi bằng giao diện đồ hoạ
ST-Link Driver	Mới nhất từ st.com	Nạp firmware và debug qua SWD
Arduino IDE hoặc ESP-IDF	2.2+ / 5.0+	Lập trình ESP32

Công cụ	Phiên bản tối thiểu	Mục đích
Serial Terminal (PuTTY, Tera Term)	—	Quan sát log UART khi debug

5.2. Quy trình cấu hình CubeMX

1. Tạo project mới, chọn MCU `STM32F401CCU6`.
2. Tab **SYS**: Debug → Serial Wire (để nạp và debug được).
3. Tab **RCC**: HSE → Crystal/Ceramic Resonator (nếu board có thạch anh ngoài, thường là 8MHz hoặc 25MHz). Cấu hình Clock Tree: SYSCLK = 84 MHz, APB1 = 42 MHz (Timer APB1 ×2 = 84 MHz), APB2 = 84 MHz.
4. Cấu hình **GPIO PA4**: Output Push-Pull, No Pull, Low Speed (chân DHT22).
5. Cấu hình **TIM2**: Channel 1 → PWM Generation CH1. Prescaler = 47, Counter Period = 19999. Trong PWM Generation: Pulse = 1500 (giá trị khởi đầu 90°). Auto-reload preload = Enable.
6. Cấu hình **TIM11**: Activated. Prescaler = 83, Counter Period = 65535. Không cần bật ngắt.
7. Cấu hình **USART1**: Mode Asynchronous, Baudrate 115200, 8N1. Tab NVIC Settings: bật `USART1 global interrupt`.
8. Cấu hình **USART2**: Mode Asynchronous, Baudrate 9600, 8N1.
9. Project Manager → Toolchain: STM32CubeIDE. Code Generator: chọn *Generate peripheral initialization as a pair of .c/.h files per peripheral*.
10. Nhấn **Generate Code**.

5.3. Các bước chạy dự án lần đầu

1. Copy hai thư mục `App/` và `App/Libraries/` vào trong `Core/`.
2. Vào **Project** → **Properties** → **C/C++ General** → **Paths and Symbols** → **Includes**, nhấn **Add...** và thêm:
   - `../Core/App`
   - `../Core/App/Libraries/DHT22`
   - `../Core/App/Libraries/DFPlayer`
3. Mở `main.c`, trong section `/* USER CODE BEGIN Includes */` thêm `#include "ChuongTrinhChinh.h"`.

4. Trong `main()`, sau khối `MX_*_Init()`, thêm đúng một dòng `ChuongTrinh_Init();`. Trong vòng `while(1)` thêm `ChuongTrinh_Loop();`. Không thêm gì khác.
5. Build (Ctrl+B). Nếu có lỗi "undefined reference" → kiểm tra lại Include Paths. Nếu lỗi "multiple definition" → kiểm tra header guard trong file `.h`.
6. Kết nối ST-Link, nạp firmware (F11).
7. Mở Serial Terminal trên máy tính, kết nối chân TX-RX của STM32 với USB-TTL, baudrate 115200 để kiểm tra dữ liệu gửi ra.

5.4. Các lỗi thường gặp và cách khắc phục

Triệu chứng	Nguyên nhân có thể	Cách khắc phục
DHT22 luôn trả về 0 (đọc lỗi)	Chưa gọi `HAL_TIM_Base_Start(&htim11)`; thiếu điện trở kéo lên; đấu nhầm chân	Thêm lệnh start timer trong `ChuongTrinh_Init()`; gắn trở 4.7kΩ giữa DATA và VCC; kiểm tra lại sơ đồ
UART nhận được byte nhưng frame không bao giờ ready	Không re-arm `HAL_UART_Receive_IT`; CRC tính sai ở ESP32; state machine mắc kẹt	Đảm bảo dòng re-arm ở cuối callback; in log byte nhận được để so sánh; thêm timeout reset về `WAIT_HEADER_1` sau 100ms không có byte mới
Servo rung giật, không mượt	Nguồn yếu; chu kỳ PWM sai (không phải 50Hz); có `HAL_Delay` chen vào trong loop	Cấp nguồn 5V 1A riêng cho servo; kiểm tra lại TIM2 Prescaler=47, Period=19999; rà soát tất cả `HAL_Delay` trong code
Toàn bộ hệ thống treo sau vài phút	Biến chia sẻ với ISR chưa khai báo `volatile`; vòng while chờ không có timeout	Thêm `volatile` cho `rx_byte`, `frame_ready`, `rx_state`; kiểm tra mọi `while` trong driver đều có biến timeout
Nạp code được nhưng không chạy	Chưa cấu hình Clock Tree đúng; HSE không hoạt động	Mở CubeMX, chọn lại HSE crystal, regenerate; hoặc dùng HSI làm clock source để test

5.5. Quy trình Debug khoa học

1. **Test từng module riêng trước khi tích hợp.** Viết một `main.c` tối giản chỉ để test DHT22, in kết quả qua UART. Sau khi driver chạy ổn mới tích hợp vào hệ thống lớn.
2. **Thêm log đường truyền.** Trong callback UART, gửi ngược byte vừa nhận ra một UART khác (hoặc qua ITM Trace) để kiểm tra đã nhận đúng chưa.
3. **Dùng breakpoint có điều kiện.** Đặt breakpoint bên trong `Process_Command()` với điều kiện `command == 0x02` để chỉ dừng khi nhận đúng lệnh servo.
4. **Quan sát tín hiệu bằng oscilloscope hoặc logic analyzer.** Đây là bước không thể thiếu khi debug giao thức timing-critical như DHT22. Một chiếc logic analyzer USB giá rẻ (như Saleae clone) đủ để xem được 40 bit trả về từ DHT22.
5. **Bật Watchdog (IWDG).** Sau khi firmware ổn định, bật Independent Watchdog để hệ thống tự reset nếu bị treo. Đây là lớp bảo vệ cuối cùng.

5.6. Checklist nghiệm thu cuối cùng

Trước khi bàn giao dự án, hãy tự kiểm tra theo từng mục:

Tổ chức code

- `main.c` dưới 100 dòng logic (không tính phần CubeMX generated)
- Mỗi module có cặp `.c` + `.h` riêng
- Tất cả file `.h` có header guard
- Hàm nội bộ đã khai báo `static`
- Biến chia sẻ với ngắt đã khai báo `volatile`

Tầng phần cứng

- Không có `HAL_Delay()` trong driver timing-critical
- Timer µs đã khởi động (`HAL_TIM_Base_Start`) trước khi dùng
- Có timeout chống treo ở mọi vòng `while` chờ tín hiệu

Giao tiếp UART

- Frame có Header + Length + CRC đầy đủ
- State machine có reset khi gặp byte sai
- Callback ngắt KHÔNG chứa logic dài
- Có re-arm `HAL_UART_Receive_IT` cuối callback

- Buffer có kiểm tra giới hạn để chống overflow

Logic ứng dụng

- Mọi task định kỳ dùng `HAL_GetTick()`, không `HAL_Delay()`
- Biến trạng thái có giá trị khởi tạo rõ ràng
- Giá trị giới hạn an toàn (góc servo 60-120) có `#define` hoặc `const`
- Đã test từng module riêng trước khi tích hợp

LỜI KẾT

Lập trình embedded chuyên nghiệp không phải là viết code *hoạt động được*, mà là viết code có thể **đọc hiểu được** sau 6 tháng quay lại, **tái sử dụng được** cho dự án khác, **mở rộng được** mà không phá code cũ, và **debug được** khi có lỗi production.

Kiến trúc module, máy trạng thái UART và tư duy non-blocking mà bạn vừa học trong tài liệu này chính là nền móng cho tất cả những điều đó. Hãy áp dụng vào dự án E-Nanny ngay từ hôm nay, và bạn sẽ thấy sự khác biệt rõ rệt khi chuyển sang dự án tiếp theo.

*Tài liệu biên soạn cho dự án **E-Nanny – Smart Baby Monitor***
STM32F401CCU6 + ESP32 | Phiên bản 1.0 | 2026